Cairo University

Faculty of Computers and Artificial Intelligence

CS251
Introduction to Software Engineering

Personal Budget App

Software Design Specifications
Version 1.0

#	Student Name	Student ID	Email	Section
1	Ibrahem Ahmed Ezzat	20240004	20240004@stud.fci-cu.edu.eg	28
2	Mohamed Abdullah Abdulhafiz	20240515	20240515@stud.fci-cu.edu.eg	28
3	Hesham Mohamed Fathy	20240649	20240649@stud.fci-cu.edu.eg	28
4	Youssef Sayed Ali	20240692	20240692@stud.fci-cu.edu.eg	17

May, 2026

1. Project Introduction & Architecture

Architecture Overview

The system is built using the Model-View-Controller (MVC) architectural pattern. This design ensures a clean separation of concerns, making the application scalable and easy to maintain.

- **Model:** Defines the data structures (User, Transaction, Budget).
- **View:** Handles the JavaFX user interface.
- **Controller:** Manages the communication between the UI and the business logic.

controllers	5/6/2026 4:05 PM	File folder
data	5/6/2026 5:25 PM	File folder
Documentation	5/7/2026 12:04 PM	File folder
models	5/6/2026 4:13 PM	File folder
out	5/6/2026 4:24 PM	File folder
services	5/6/2026 4:19 PM	File folder
ui	5/6/2026 5:28 PM	File folder
app.iml	5/6/2026 4:32 PM	IML File
Main.java	5/6/2026 5:27 PM	IntelliJ IDEA

"Our project structure follows the Clean Architecture approach, separating data, logic, and UI into distinct packages for better maintainability."

2. The Model Layer (Data Blueprints)

Models are POJOs (Plain Old Java Objects) that represent the core entities of our application. They store the state of the system.

Class	Responsibility
User	Stores authentication details (username/password) and monthly salary.
Budget	Tracks spending limits and actual spending per category (e.g., Food).
Goal	Tracks financial targets (e.g., Save for Travel) and current savings.
Transaction	Records individual spending or income entries.

Budget.java

```java
package app.models;

/**
 * Represents a budget assigned to a specific user and category.
 * It tracks the spending limit and the amount already spent.
 */
public class Budget {  no usages  Hisham +1

    /** The ID of the user who owns this budget */
    private int userId;  2 usages

    /** The category of the budget (e.g., Food, Transport) */
    private String category;  2 usages

    /** The maximum allowed spending for this category */
    private double limit;  2 usages

    /** The amount already spent from this budget */
    private double spent;  3 usages

    /**
     * Constructs a new Budget object.
     *
     * @param userId   the ID of the user
     * @param category the budget category
     * @param limit    the maximum allowed spending
     */
```

Activate Windows

3. Business Logic: Service Layer

The Service layer contains the "Brain" of the application. It performs complex calculations and enforces system rules before data is saved.

BudgetService.java Logic

This service calculates spending percentages and triggers console/UI warnings based on thresholds:

- **Safe Zone:** Spending is less than 80% of the limit.
- **Warning Zone:** Spending exceeds 80% (Alert: "Near Limit").
- **Critical Zone:** Spending exceeds 100% (Alert: "OVER BUDGET").

BudgetService.java

```java
package app.services;

import app.data.FileDatabase;
import java.util.*;

/**
 * Service class responsible for managing user budgets.
 * It allows creating budgets and updating them based on spending,
 * while tracking limits and warning when thresholds are reached.
 */
public class BudgetService {  5 usages  Hisham +1

    private static final String FILE_NAME = "budgets.txt";  3 usages

    /**
     * Creates a new budget for a user and saves it to the database file.
     *
     * @param id the ID of the user
     * @param category the budget category (e.g., Food, Transport)
     * @param limit the maximum allowed spending for this category
     */
    public void create(int id, String category, double limit) {  Hisham +1

        FileDatabase.save(FILE_NAME,
                data: id + "," + category + "," + limit + ",0");

        System.out.println("Budget Created ✅");
```

4. Data Persistence (FileDatabase)

Since we use flat-file storage (TXT/CSV), the FileDatabase class acts as our repository layer, handling all disk I/O operations.

- **save():** Appends new records to the end of the text file.
- **read():** Loads text files and splits data by commas for the application to use.
- **overwrite():** Updates existing lines, used when budget spending changes.

FileDatabase.java

```java
package app.data;

import java.io.*;
import java.util.*;

/**
 * Utility class responsible for handling basic file-based data storage operations.
 * It provides methods for saving, reading, and overwriting data in text files
 * located in the application's data directory.
 */
public class FileDatabase {  16 usages  Hisham +1

    /**
     * Builds the full file path for a given filename inside the data directory.
     *
     * @param file the name of the file
     * @return the full path to the file
     */
    private static String buildPath(String file) {  3 usages  Hisham +1
        return "data/" + file;
    }

    /**
     * Appends a new line of data to the specified file.
     *
     * @param file the file name
     * @param data the data to be saved
```

```java
public class FileDatabase {  16 usages  Hisham +1
    public static void save(String file, String data) {  4 usages  Hisham +1
        try (FileWriter fw = new FileWriter(buildPath(file), append: true)) {
            fw.write( str: data + "\n");
        } catch (IOException e) {
            System.out.println("Save Error: " + e.getMessage());
        }
    }

    /**
     * Reads all data from a file and returns it as a list of string arrays.
     * Each line is split by comma to separate fields.
     *
     * @param file the file name
     * @return a list of string arrays representing file rows
     */
    public static List<String[]> read(String file) {  5 usages  Ibrahem Ahmed +1
        List<String[]> list = new ArrayList<>();

        try (BufferedReader br = new BufferedReader(new FileReader(buildPath(file)))) {
            String line;
            while ((line = br.readLine()) != null) {
                list.add(line.split( regex: ","));
            }
        } catch (IOException e) {
            System.out.println("Read Error: " + e.getMessage());
```

5. Code Documentation (JavaDoc)

Professional Standard

We followed the industry-standard JavaDoc format to ensure our code is self-explanatory and professional.

Key JavaDoc Tags Used:

- /** ... */: Block comments for classes and methods.
- @param: Describes each input parameter.
- @return: Explains the data being returned.

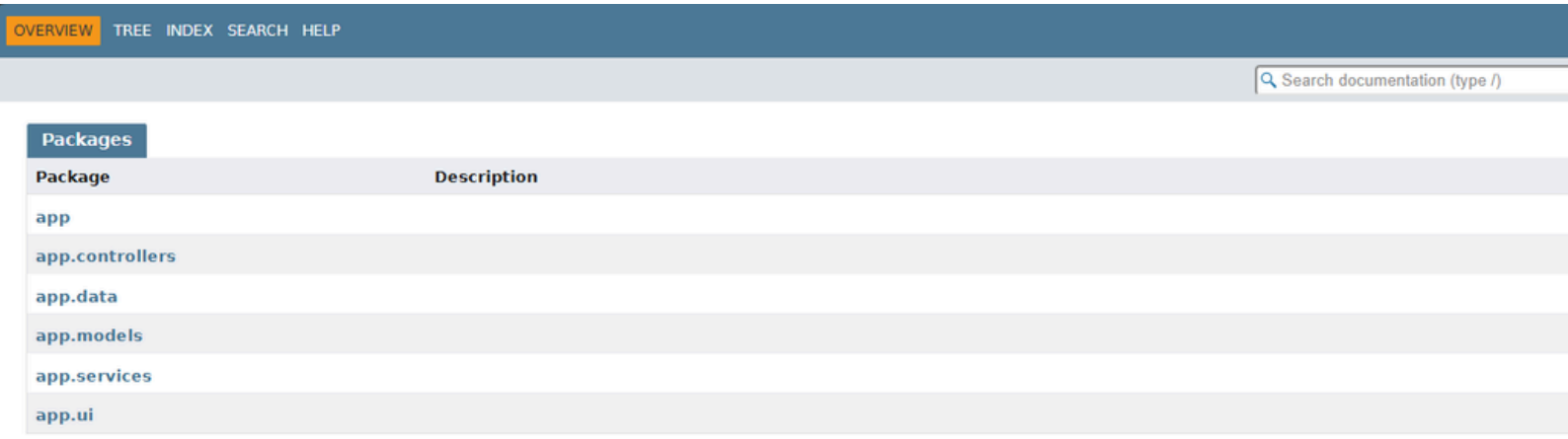
GoalService.java

```java
package app.services;

import app.data.FileDatabase;

/**
 * Service class responsible for managing user financial goals.
 * It allows creating goals and contributing savings toward them,
 * while tracking progress and completion status.
 */
public class GoalService {  3 usages  Hisham +1

    private static final String FILE_NAME = "goals.txt";  2 usages

    /**
     * Creates a new financial goal for a user and saves it to the database file.
     *
     * @param id the ID of the user
     * @param name the name of the goal (e.g., Buy a car, Travel)
     * @param target the target amount required to achieve the goal
     */
    public void create(int id, String name, double target) {  Ibrahem Ahmed +1

        FileDatabase.save(FILE_NAME,
                data: id + "," + name + "," + target + ",0");

        System.out.println("🎯 Goal Created");
    }
```

6. Generated Technical Documentation

Using the automated JavaDoc engine, we converted our in-code comments into a professional HTML documentation website.

- Package Navigation: Users can browse classes by their MVC layer.
- Method Details: Lists every function, its purpose, and its parameters.



7. Style Compliance & Readability

Our code complies with the Oracle Java Coding Style Manual, ensuring that it is readable by any developer.

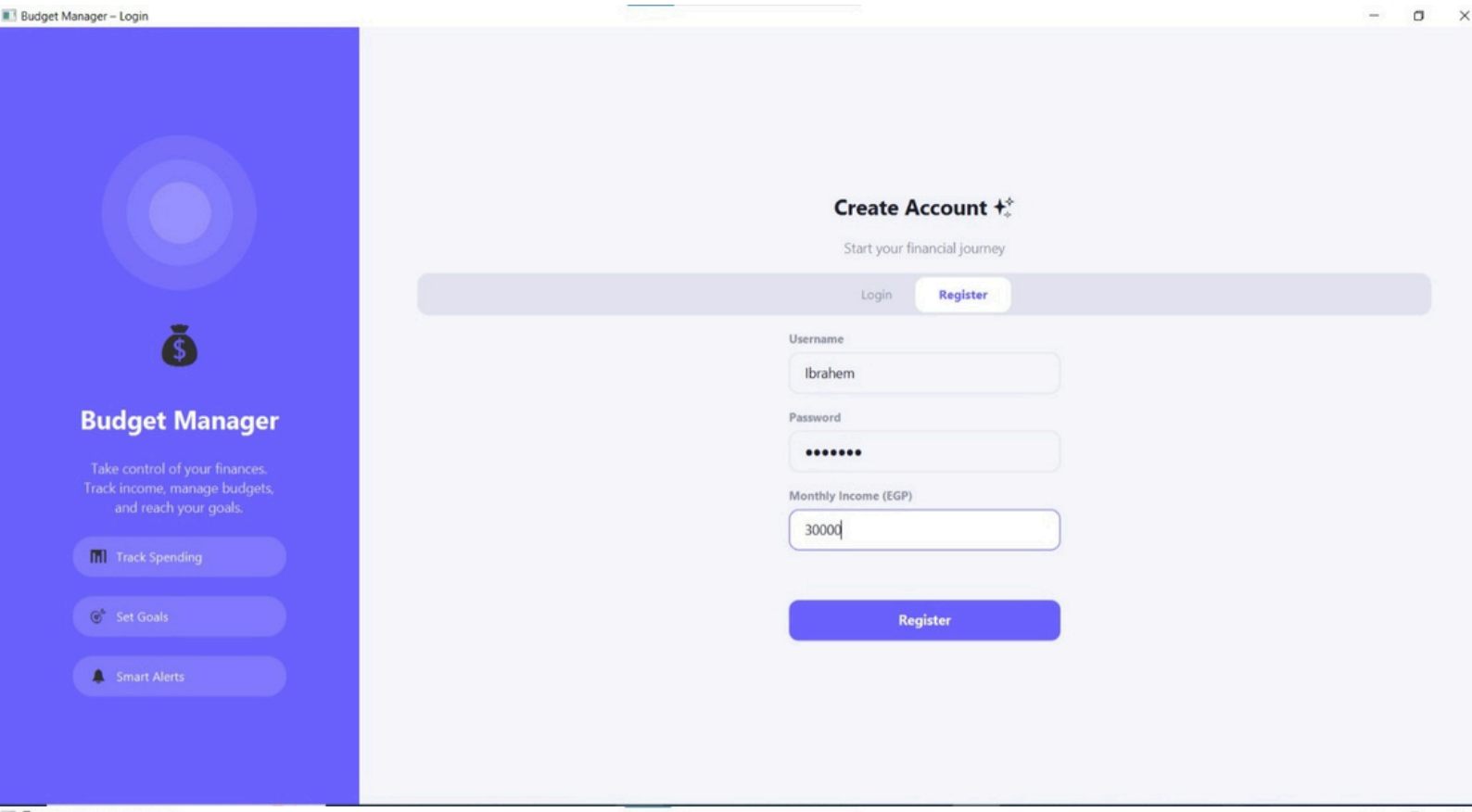
- **PascalCase:** Used for all Class names.
- **camelCase:** Used for variables and method names.
- **Indentation:** Consistent 4-space indentation for logical blocks.

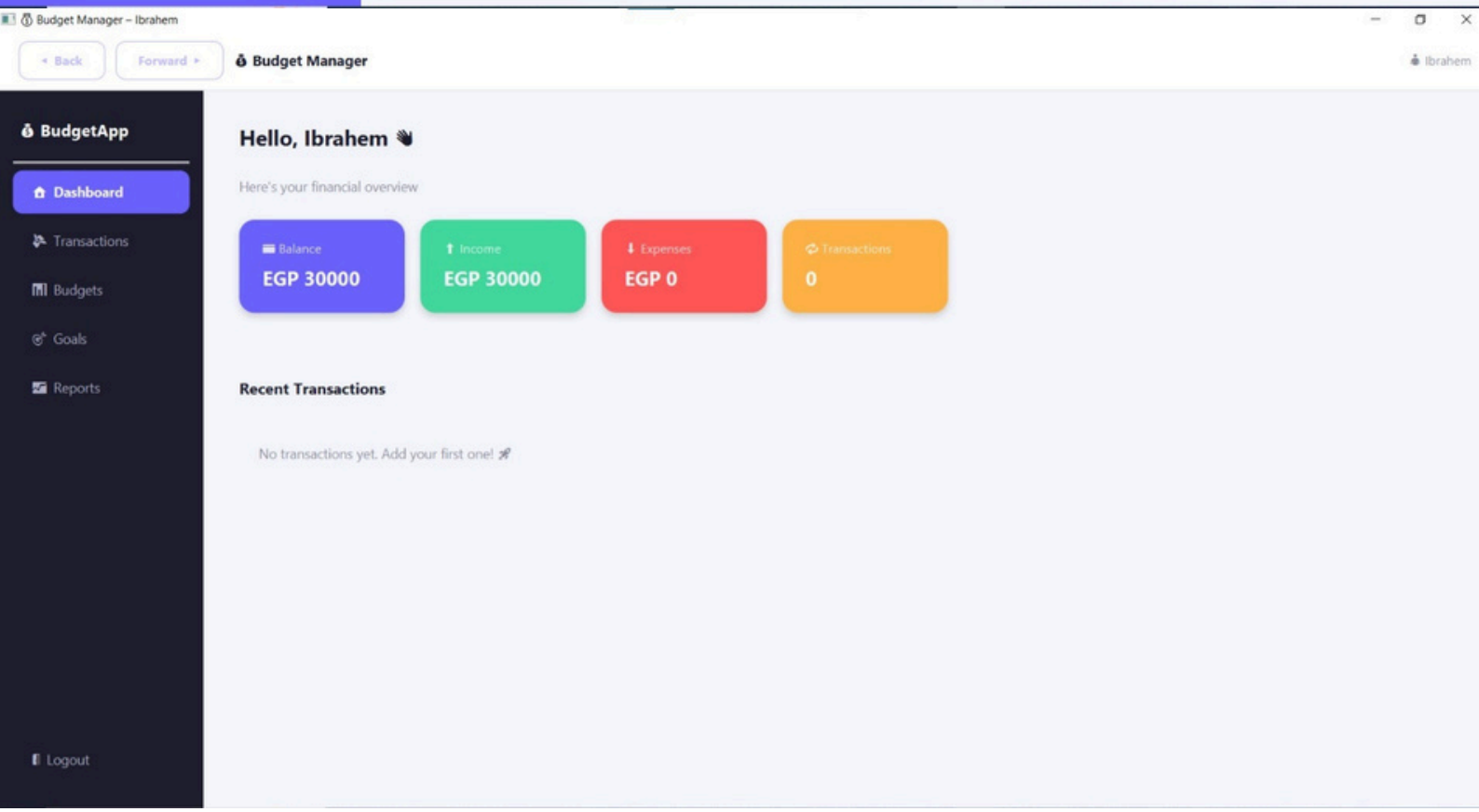
TransactionController.java

```java
package app.controllers;

import app.services.TransactionService;

/**
 * Controller class responsible for handling transaction-related requests
 * and delegating them to the TransactionService.
 * It acts as an interface between the presentation layer and business logic.
 */
public class TransactionController {  3 usages  Hisham +1

    /**
     * Service instance used to manage transaction operations.
     */
    private TransactionService transactionService = new TransactionService();  1 usage

    /**
     * Adds a new financial transaction for a user.
     *
     * @param userId the ID of the user
     * @param amount the transaction amount
     * @param category the category of the transaction (e.g., Food, Transport)
     */
    public void add(int userId, double amount, String category) {  Hisham +1
        transactionService.add(userId, amount, category);
    }
}
```

8. User Interface (JavaFX)

The UI provides an interactive way for users to manage their finances. It communicates with the Controllers to perform actions.





9. Feature: Validation Logic

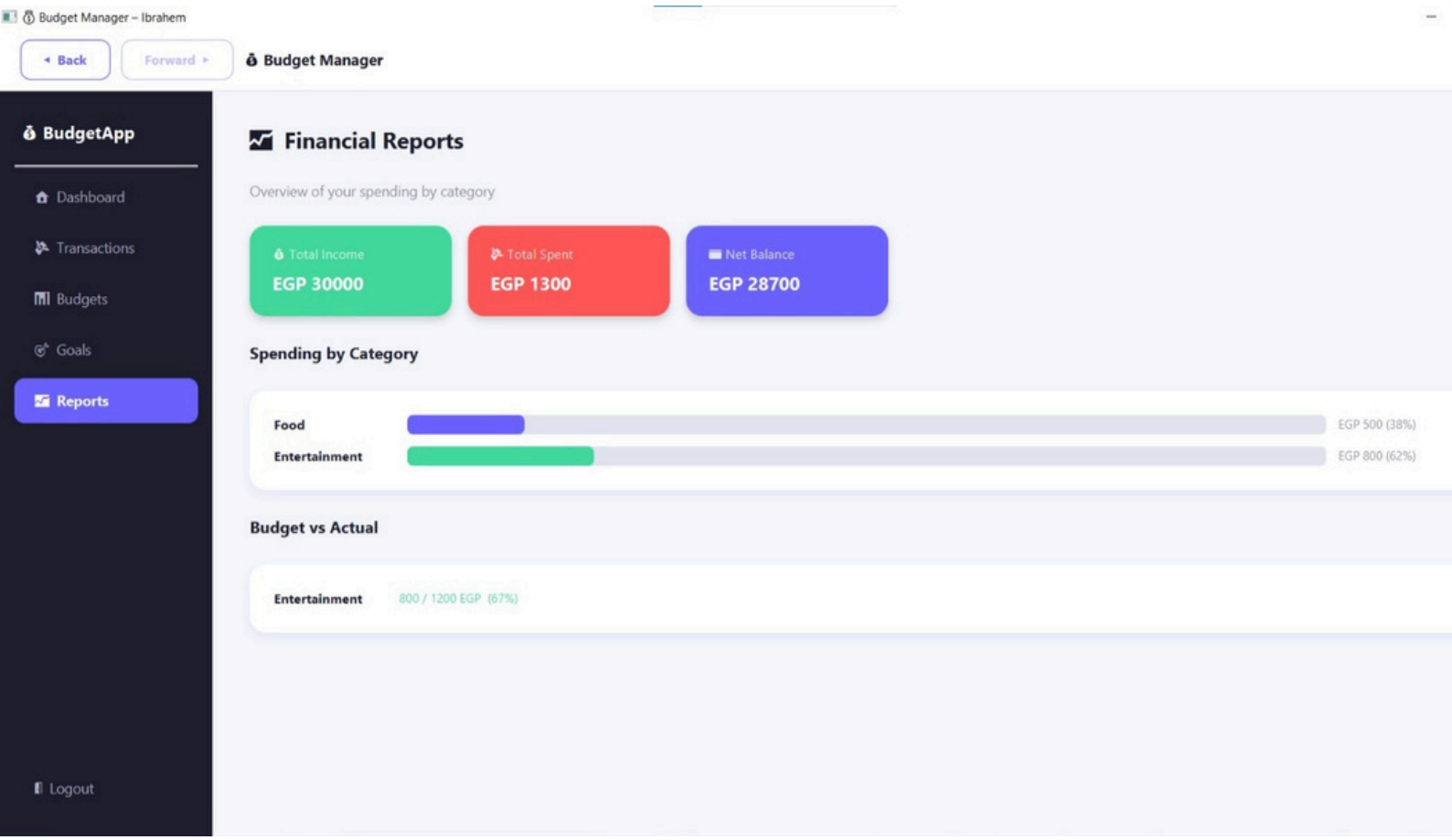
To prevent bad data, we implemented validation logic. For instance, a budget cannot be set higher than the user's monthly salary.

BudgetUI.java

```java
public class BudgetUI {  1 usage  Ibrahem Ahmed +1
    public void show() {  Ibrahem Ahmed +1
        // Handle save action
        save.setOnAction( ActionEvent e -> {
            try {
                double l = Double.parseDouble(limit.getText());

                // Validate budget does not exceed salary
                if (l > user.getSalary()) {
                    msg.setText("Budget > Salary ❌");
                    return;
                }

                // Create budget using controller
                controller.create(user.getId(), category.getText(), l);
                msg.setText("Budget Created ✅");

            } catch (NumberFormatException ex) {
                msg.setText("Invalid Input!");
            }
        });

        // Navigate back to main menu
        back.setOnAction( ActionEvent e -> mainMenu.show());

        // Layout container
        VBox root = new VBox( v: 10, category, limit, save, back, msg);
```

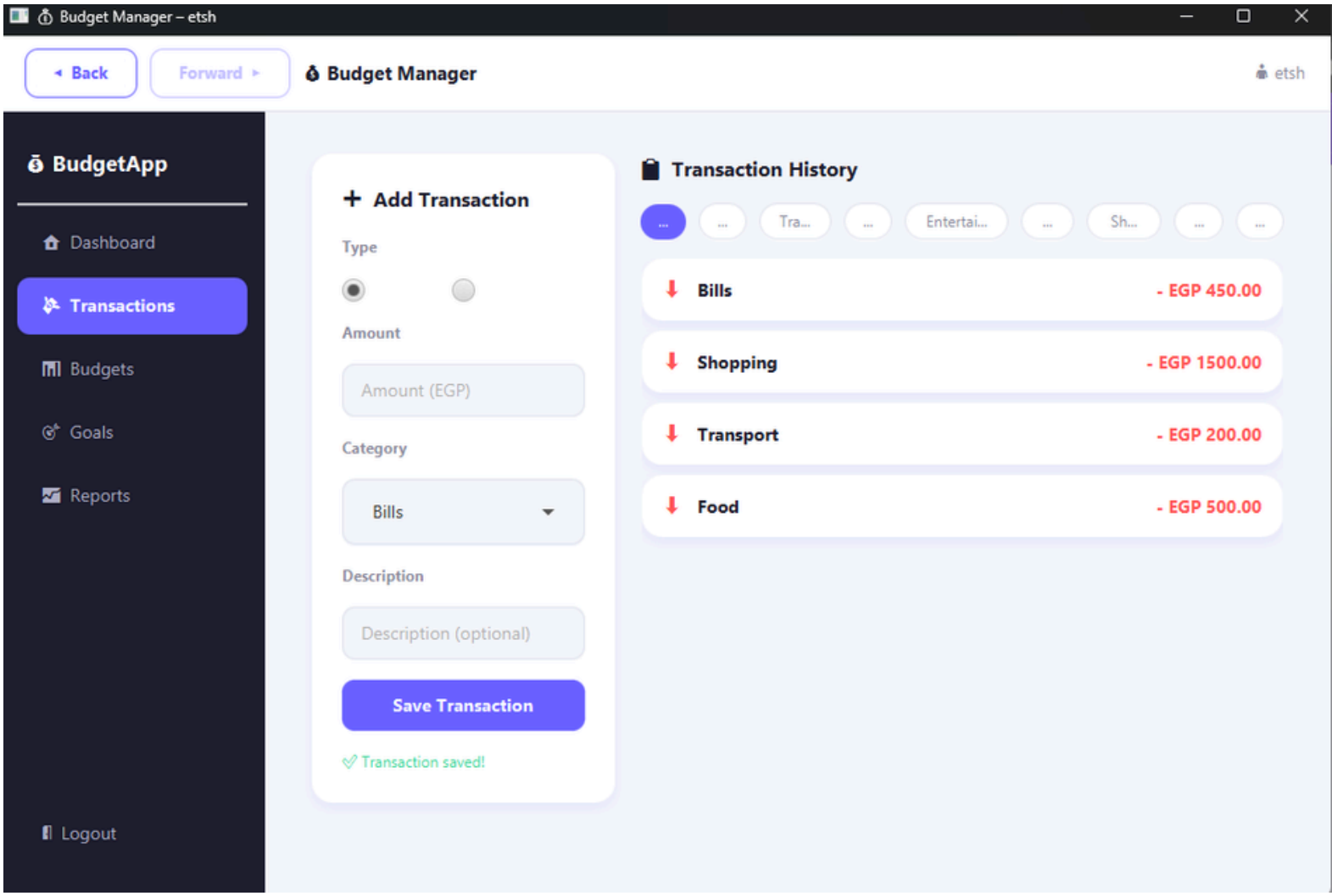
10. Feature: Financial Reporting

The reporting module aggregates all transaction data and provides a summarized breakdown of spending per category.



11. Conclusion & Future Roadmap

The Personal Budget Management System successfully implements the MVC architecture with professional documentation and clean code standards.

Roadmap:

- **Database Migration:** Moving from text files to SQLite/MySQL.
- **Enhanced Security:** Implementing SHA-256 password hashing.
- **Data Visualization:** Adding Pie charts for spending analysis.